Assignment 4: Parsing and Fault Injection

Gabriel Castillo (gac232), Leo Qian (rq48), Wallace Wei (wzw4), Narayan Topalli (nvt5)

Summary

Interestingly enough, the most challenging part of the assignment was designing the AST structure itself. At the beginning of this assignment we did not do a thorough enough job in communicating, agreeing on, and specifying an AST structure. This led to quite a few problems down the line as we had to continuously redesign the AST and add or remove necessary and unnecessary methods. In all fairness, the AST most definitely requires very complex class inter-dependency and it's hard to think ahead to create a perfect AST structure that avoids any possible roadblocks down the line, but we managed to pull through in the end and complete the assignment.

An interesting design decision was not implementing the *Command* class and instead having the Rule class store Condition, Updates, and Action. Thinking back, this was one of many mistakes we made that would've improved the readability of AST. Delegating the work of *Command* and all its class invariants to *Rule* made it more difficult than it had to to maintain the class invariant of the *Rule* class, which caused quite a few headaches at times.

Specification Changes

In the spec, it was specified that we extend the Tokenizer class to ignore java-like comments. However, since all that was missing for Tokenizer was that one functionality, we opted to inject our implementation of this functionality into the Tokenizer class directly.

For testing purposes, we made several methods public, including getType() in Token class.

Additionally, we added several methods within our ParserImpl that assists in parsing left-associative AST:

- parseCondition(), parseConjunction(), and parseRelation() parses Conditions similar to the way the parse() method in the lecture notes works.
- parseExpression(), parseTerm(), and parseFactor() are identical to the parseE() function in lecture notes and parses math equations.
- parseAction() and parseUpdate() are added to parse action and update classes
- Added various consume() methods to consume a token according to its Category, Type, or a given predicate lambda. Throws a SyntaxError when it encounters an unexpected token.

For our AST, we modified AbstractNode, which was then extended by each of our node categories.

- Added findNodesOfPredicate() which searches the subtree rooted at this abstractNode and returns a list of nodes that satisfy a given predicate
- Added setParent() which sets the parent of this abstractNode to the given abstractNode
- Added abstract setChildren() which sets the children of an abstractNode to the given node list in the same order as getChildren(). This function first checks if the given list is a

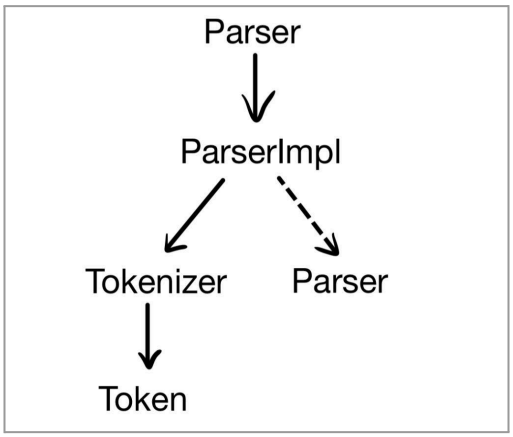
valid list of possible children, and if and only if the list passes the check will it continue to modify its children variables. It returns a boolean that states whether the list passes the check or not. We chose to implement the checks because we don't want to accidentally break a well-formed AST and/or violate the class invariant of a node.

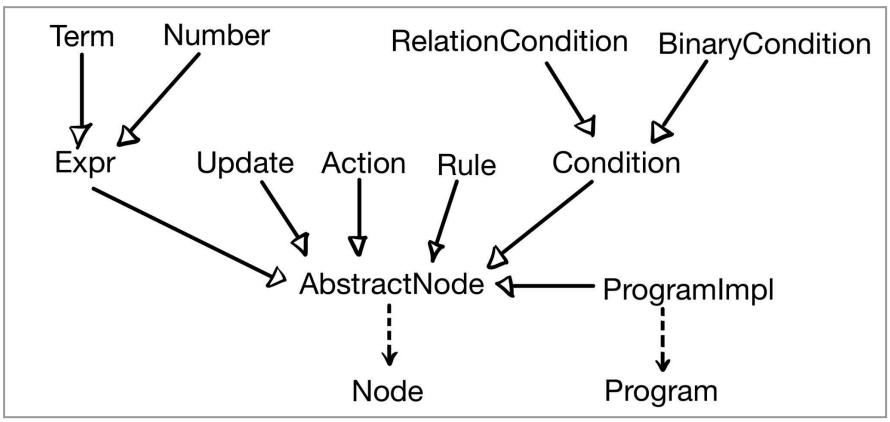
IMPORTANT NOTE: While reading through the spec, there seemed to be a conflicting statement which states that we must both 1) not print redundant parentheses and 2) be able to parse any AST's prettyPrint back into the original AST. It's impossible to satisfy both conditions since not pretty-printing any parenthesis, even redundant ones, would result in a different AST structure in our case. Therefore, we opted to ignore redundant parenthesis while printing and ignore the second requirement of the project spec. However, what we can guarantee is that for any randomly generated AST a_1, when we pretty print and parse it to a_2, and then pretty print and parse a_2 to a_3, a_2 is guaranteed to be identical to a_3. This is because a_2 will remove all redundant parenthesis, so a_3 will have no parenthesis to remove, as such, $a_3 = a_2$.

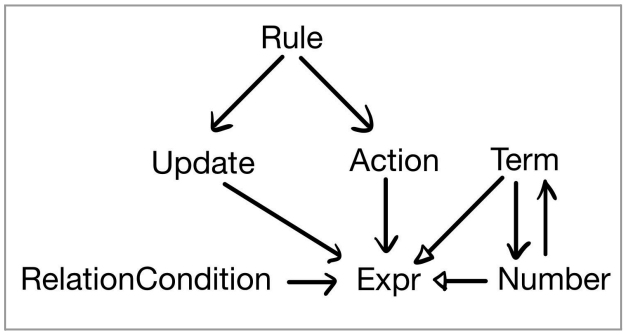
Also, for Insert() in mutation, it tries to avoid assigning double negatives inside the AST (no --6, but -(-3+6) is fine)

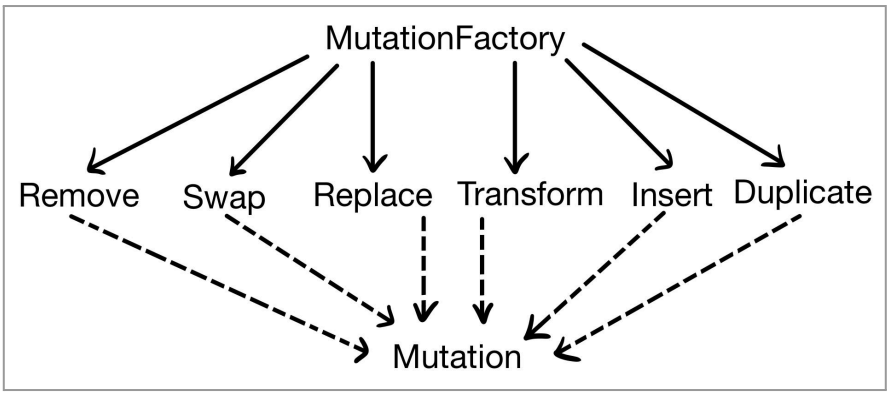
Design and Implementation

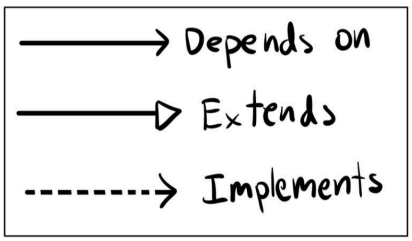
Below are the Module Dependency DiagramsNote. Note that there are 2 module dependency diagrams for the AST package, the reasoning behind this is that having 1 MDD for the entire AST package results in a diagram that is very messy.











Architecture:
In terms of architecture, we drew out a module-dependency diagram very early because this assignment contains many classes, so subtyping and dependency relationships between classes would be important to identify and organize.

While designing the MDD on the AST package, we considered what types of nodes the AST would have and created one unique class for each node type. We tried to make every class as simple as possible, diverting much of the work to the Parser package to correctly construct an AST. The reasoning behind this is described in the Work Plan section.

Update stores 2 *Expr* such that mem[expr1] := expr2, it has a second constructor that can take in a TokenType of TokenCategory.MEMSUGAR to automatically store and construct an Expr object to represent the constant value stored in the memsugar token.

Term stores 2 *Number* such that num1 op num2

BinaryCondition stores 2 *Condition* such that condition1 op condition2

RelationCondition stores 2 *Expr* such that expr1 op expr2

Action stores an ActionType and an Expr, where Expr is only not null when ActionType = serve, since serve takes in an Expr

Rule stores 1 Condition, 1 Action, and any # of Update such that Condition must not be null and Action being null implies that Update is not empty.

Factor stores a NumberType and an Expr. This class has multiple constructors that allows it to convert a variety of constructor parameters into Expr and NumberType, where Expr is an expression and NumberType stores the type of Factor.

These conditions for each class are checked within every constructor with an assert statement.

While designing the MMD for the Mutation interface section, we realized that we need 6 unique classes for the 6 unique types of Mutations. We put them inside a new package inside our AST called mutations. Creating a unique class file for each mutation allows our code to have improved readability and we are also able to access each mutation subtype outside of MutationFactory. Additionally, this allows us to have unique implementations for each and every mutation class, and after implementing the rest of our AST according to our Bottom-up implementation plan, we could easily make modifications to our mutations.

In terms of the MDD for the parsing package, we didn't make any changes to existing class structures and didn't create any new classes, since the provided classes are already well-structured and don't require any editing.

Code design:
First we need to implement Breadth First Search in order to create a way to traverse the tree. This is used on findNodeOfType() in ProgramImpl to find all types of nodes that satisfy a given predicate, and nodeAt() in AbstractNode for how we index any descendant of a given node.

For PrettyPrinting, we use a recursive call of prettyPrinting that mimics the traversal of Depth First Search to ensure the correct ordering is printed out.

We use recursive calls for size(), classInv(), and clone() method for any node to its children.

Size() sums up every size() call to its children and returns that value + 1. classInv() checks its own class invariant and returns if it satisfies its class invariant and if all its children satisfies their class invariant. Clone() calls its children's clone() method and inputs them into the parameters of its own constructor, along with any other values required in its constructor.

In terms of data structures, we are using trees with nodes that have a list of children and a reference to parent in the AST in order to facilitate traversal.

Programming:

We decided on a bottom-up implementation strategy in order to thoroughly understand the complex parsing functions and AST structure. This enables us to efficiently debug and make any improvements to our pre-existing class hierarchy early on, if needed.

Testing

Given the complexity of pretty-printing, mutations, and Parser, we decided to use a mixture of both black box and glass box testing. Glass box testing would help with ensuring the data structures and algorithms used inside of our functions work correctly under edge cases, while black box testing would help in testing more extreme edge cases and the compatibility between different modules.

In order to thoroughly test mutations, pretty-printing, and our implementation of all the different nodes in an AST, we decided to create a random AST generator class. This class takes in the maximum number of children and a number of around how many notes the randomly generated AST should have. Then it creates a random AST that follows all the classInv() methods of all the nodes in the tree, or it can also create a random version of a specific kind of node with random children. To test this class we called the random generator function for all the different kinds of nodes and called the classInv() method for each of them.

The classInv() method of each kind of AST node is extremely important. A good implementation of this method allows us to test that any given AST is well formed. In every AST node, in this function we check that the parents and children of the node are of the correct type, and that the current node follows the certain specifications explained in the project specification. Finally, we then call the classInv() method of any child nodes, as this method should not return true if the subtree up to this node is not well formed, and it cannot be well formed without ensuring that all children follow their classInv() method. For example for the Term class we check that it has an Expr on the left and right, that its parent is either an Action, Update, RelationCondition or Expr, that its operation category is in TokenCategory.ADDOP or TokenCategory.MULOP, that its value is equal to the operation as a string, and that the classInv() of its child expressions is true.

Additionally, in an attempt to prevent our AST's class invariants from being violated in the first place, we injected several assert statements into many functions and methods within our code, ensuring their preconditions are all satisfied before letting the code run.

Together, the RandomASTGenerator with the well implemented classInv() class will let us test all mutation types on a random AST and ensure that the resulting AST is still a well-formed tree!

We used manual and random test cases for testing pretty-printing:

- For manual test cases, we used two HashMaps that store a library of pre-made strings

(key = input, value = expected-output), one for expressions and one for conditions. We would iterate through the library for keys and call parseExpression() and parseCondition() from the ParserImpl class to generate expr and condition instances and assert their pretty-printed string must be equal to the value that the library maps to.

- For random test cases, we used our RandomASTGenerator class to generate random well-formed AST and pretty-print them. We then tested that the parser doesn't throw any syntax errors when parsing the resulting pretty-printed string. Additionally, we stated that when the parsed AST is pretty-printed and parsed again, the results of those two operations should be equal to the parsed AST. This is explained in more detail near the end of our specification change.

- To test for false negatives in random test cases, we wrote a method swapTokens() which takes in a string from a well-formed AST's prettyprint, converts it into an array of Tokens, and then randomly swaps around tokens which intentionally creates syntax errors. We turn the resulting gibberish into a string and attempt to parse it, expecting a very high likelihood of failure.

For mutation tests, we use RandomASTGenerator to generate the test cases. We assert that the classInv() method for the root of the AST must be true both before and after the mutation occurs. Additionally, for Duplicate Insert and Remove, we assert that the mutated AST must be different from the un-mutated AST, we didn't do this for *every* mutation, because

- Swap could swap 2 identical children, that is, $child_1$.equals($child_2$), which results in identical AST before and after mutation

- Transform could transform the node into another node identical to itself. For example, the transform mutation can theoretically replace an Action node of Action Type *forward* with a brand new Action node of Action Type *forward*, we didn't enforce that the new node created by Transform **must be** different from the original node because since mutations are allowed to fail, we figured there was no harm in the mutation not changing the AST at all, given a low enough probability of this occurring.

- Replace could replace a node with its own subtree. This is mainly for simplicity's sake, since the canApply() method is expected to return if the given node can be applied just on the node itself, we assumed that replacing the node with it's own subtree is a valid mutation and, thus, any node is valid for a replace mutation. Otherwise if Replace *cannot* replace itself, there would be a lot more computation involved. Additionally, this scenario where the node replaces itself is relatively rare, since there usually are many nodes of the same type for any node in the AST (except ProgramImpl).

For Swap Transform and Replace, instead of asserting that the mutated and un-mutated AST must be different, we simply printed the output to console and manually checked if the mutation isn't returning identical AST trees with high proportionality.

We also added simple tests to test the basic functionality of each of our mutations, so that we could make sure they are working as expected. It was difficult to make these tests more complex because of the number of edge cases that existed.

- For Replace we checked that the mutated node is of the same category as the old node.
- For Remove we checked that the node's old indexed spot is now used by another node.
- For Swap we checked that the node's new children and old children contain each other.

- For Transform we checked that the mutated node has the same category and children as the old node.
- For Duplicate we checked that the mutated node has one additional child.
- For Insert we checked that the node is contained within the inserted node's children.

While random and manual testing are written,

Work Plan

Since we have 2 pairs of partners that worked with each other on previous assignments, we figured splitting the two packages (AST and Parser) between the two partners would work out well.

Narayan (nvt5) and Wallace (wzw4) are working on the AST

Gabriel (gac232) and Leo (rq48) are working on the Parser

This puts some pressure on Narayan and Wallace to finish AST fast and correctly, since the Parser is very dependent on the AST being complete to begin testing. However, since our design for AST diverts most of the work to the parser, we figured AST will finish much sooner than parser such that when parser is complete and goes on to testing, AST will already have almost if not already finished the testing phase.

More specifically, according to the a4 specs doc, here is our task assignment:

1. **(Leo)** Alter Tokenizer to support end-of-line comments
2. **(Leo & Narayan)** Implement the main method and command-line interface.
3. **(Leo & Gabriel)** Design and implement a class hierarchy for representing abstract syntax trees.
4. **(Leo & Gabriel)** Implement the Parser interface to generate abstract syntax trees.
5. **(Leo & Wallace)** Implement pretty-printing functionality as methods on AST nodes.
6. **(Gabriel)** Create RandomASTGenerator class.
7. **(Leo & Gabriel)** Design and write thorough test cases for parser
8. **(Narayan & Wallace & Leo)** Implement classes to perform fault injection (mutations).
9. **(Leo & Wallace)** Design and write thorough test cases for mutations.
10. **(Gabriel)** Solve the written problems (which are reviewed by all).

Known Problems

In extremely rare scenarios, the testParseRandomASTSyntaxError() test in ParserTest will fail. This is because it’s theoretically possible for swapTokens() method to return a valid AST string that the ParserImpl can parse without throwing a syntax error, since its swapping around arbitrary tokens 10 times it could swap the same 2 tokens back and forth, or swap 2 tokens that happen to be completely valid replacement for each other, etc. This has not been observed yet after running hundreds of thousands of tests, however since it is theoretically possible for the test case to fail, we thought it’d be best to put this in the Known Problems section.

Comments

None for now.